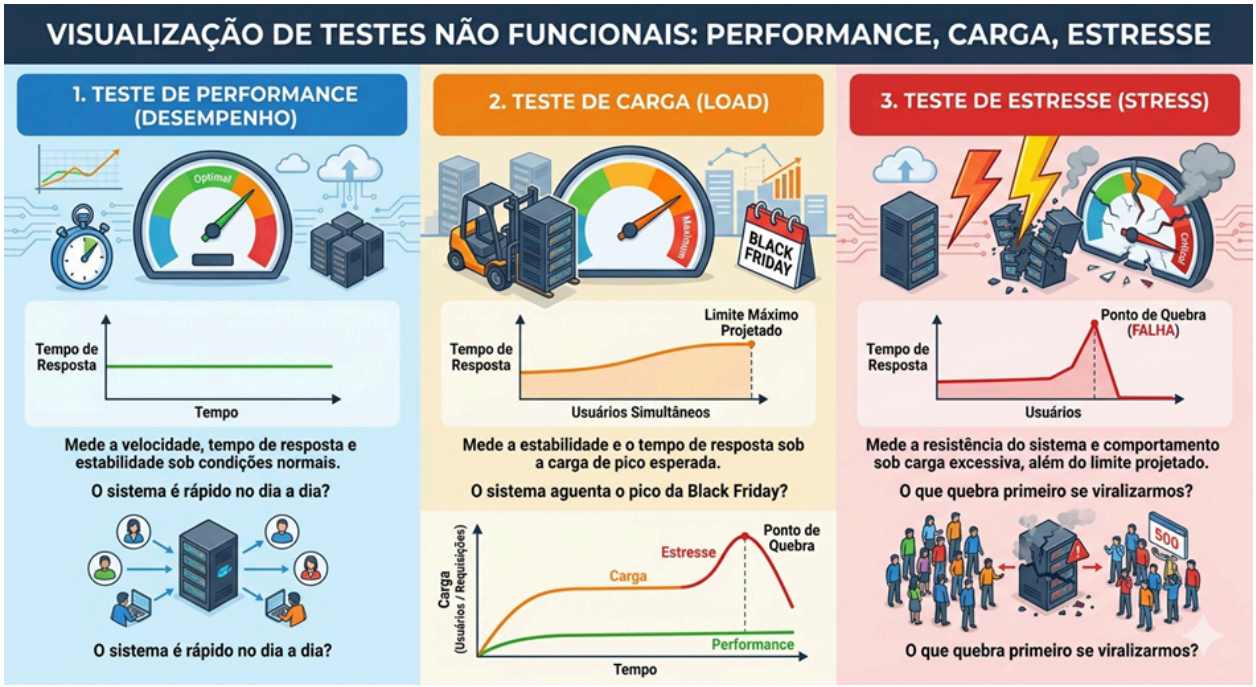
Testes Não Funcionais

Validam como o sistema opera (velocidade, estabilidade, escalabilidade), e não o que ele faz. São essenciais para garantir boa experiência do usuário e prevenir quedas, identificando gargalos antes que afetem o negócio.



Por que realizar

Performance

Garante tempos de resposta rápidos, prevenindo latência.

Carga

Valida se o sistema suporta o tráfego esperado (ex: 10.000 usuários/segundo).

Estresse

Determina o limite de capacidade e como o sistema falha sob carga extrema (resiliência).

Como realizar

Definir cenários

Estabeleça metas baseadas em produção (número de usuários, TPS).

Usar ferramentas

Utilize ferramentas como Apache JMeter, k6 ou BlazeMeter.

Monitorar

Analise consumo de CPU, memória e tempo de resposta.

Simular

Aumente gradualmente a carga (teste de carga) ou exceda os limites (teste de estresse).

Diferença entre Teste de Carga e Teste de Estresse

O teste de carga avalia o sistema sob condições esperadas de uso (tráfego normal ou de pico) para garantir estabilidade.

O teste de estresse leva o sistema além de seus limites, aplicando cargas extremas para identificar:

- ponto de ruptura
- comportamento sob falha
- capacidade de recuperação

Métricas de Performance

Latência

Tempo de resposta de uma requisição (ex: milissegundos).

Throughput

Quantidade de transações processadas por unidade de tempo (ex: transações por segundo).

Equilíbrio é essencial: alto throughput pode aumentar a latência.

Ferramentas de Teste de Carga

Ferramentas como k6, Apache JMeter e Gatling simulam tráfego intenso para validar desempenho sob carga. Permitem identificar:

- gargalos
- tempo de resposta
- capacidade máxima

Podem ser código aberto, SaaS ou freemium.

Gargalo de Desempenho em Software

Um gargalo é um componente que limita a performance do sistema, causando lentidão, alta latência e má experiência do usuário. Geralmente ocorre em:

- CPU
- Memória
- I/O de disco
- Rede

Acontece quando uma parte do sistema não acompanha a velocidade das demais.

Perguntas e Respostas

1. Como saber se o problema está na aplicação ou na infraestrutura?

Observe as métricas: CPU alta indica problema na aplicação (processamento excessivo); CPU baixa com lentidão indica problema externo (banco, rede, API). Para testar, desconecte dependências (ex: simule o banco). Se o desempenho melhorar, o problema não está na aplicação.

2. O sistema funciona com 1.000 usuários, mas quebra com 1.100. O que isso significa?

Existe um limite oculto, como limite de conexões, limite de threads ou fila cheia. O sistema não escala de forma gradual — ele falha abruptamente.

3. Qual a diferença entre teste de carga e teste de estresse?

Carga: uso normal (ex: 1.000 usuários). Estresse: acima do limite (ex: 5.000 usuários).

4. Por que funciona no teste, mas falha em produção?

Porque o ambiente de teste não é igual ao de produção. Principais motivos:

- menos dados
- menos usuários
- servidores diferentes
- cache limpo

5. A média de resposta está boa, mas usuários reclamam. Por quê?

Porque a média pode esconder problemas. Exemplo: 90% das requisições são rápidas, mas 10% são muito lentas. A média parece boa, mas parte dos usuários tem experiência ruim. O ideal é analisar P95 e P99.

Exemplos de Testes

1. Teste de Performance (Tempo de Resposta)

Cenário

API de login deve responder rapidamente.

Como testar

Simular requisições contínuas com k6 ou Apache JMeter. Medir tempo médio, máximo e percentis.

Objetivo

Garantir tempo de resposta dentro do SLA (ex: < 500 ms).

2. Teste de Carga (Load Test)

Cenário

Plataforma de streaming com 10.000 usuários simultâneos.

Como testar

Simular usuários acessando vídeos simultaneamente, aumentando gradualmente até atingir a carga esperada.

Objetivo

Verificar estabilidade e desempenho sob carga real.

3. Teste de Estresse (Stress Test)

Cenário

Sistema de reservas com limite de 2.000 usuários.

Como testar

Ultrapassar o limite gradualmente (3k, 5k, 10k usuários) e monitorar falhas, lentidão e recuperação.

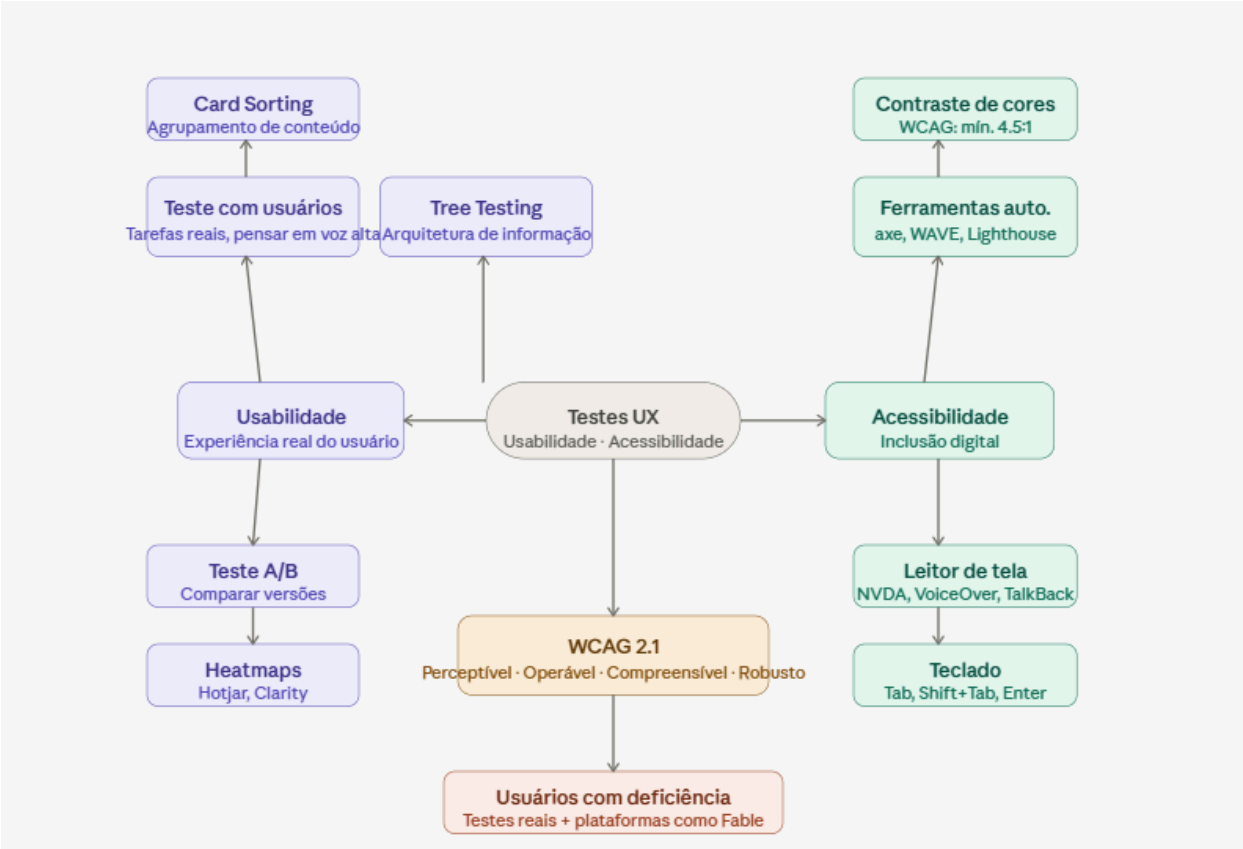
Objetivo

Identificar ponto de falha, degradação do sistema e capacidade de recuperação.

Testes de Usabilidade e Acessibilidade (UX)

O que são?

Os testes de usabilidade e acessibilidade UX avaliam a facilidade de uso e inclusão de produtos digitais, observando usuários reais realizando tarefas para identificar pontos de atrito.



Por que realizar?

Esses testes são cruciais para garantir que produtos digitais sejam intuitivos, eficientes e acessíveis a todos, incluindo pessoas que possuam alguma deficiência.

Como realizar?

Testes de usabilidade e acessibilidade são etapas essenciais no processo de design UX. As abordagens mais comuns incluem:

- Testes com usuários reais realizando tarefas específicas
- Card sorting para organizar a arquitetura de informação
- Testes A/B para comparar versões de uma interface
- Heatmaps para entender o comportamento de navegação

Para acessibilidade, o processo inclui:

- Ferramentas automáticas como axe DevTools e Lighthouse
- Verificação de navegação via teclado
- Testes com leitores de tela como NVDA e VoiceOver
- Checagem de contraste de cores seguindo os critérios da WCAG 2.1

Os quatro princípios da WCAG 2.1 são: perceptível, operável, compreensível e robusto. O ideal é combinar essas abordagens em ciclos contínuos, sempre envolvendo usuários reais, incluindo pessoas com deficiência.

Perguntas e Respostas

1. Qual a diferença entre teste de usabilidade moderado e não moderado?

No teste moderado, um facilitador acompanha o usuário em tempo real, podendo intervir e fazer perguntas. No não moderado, o usuário realiza as tarefas de forma independente, geralmente de forma remota e assíncrona. O moderado é indicado para aprofundar insights e explorar comportamentos específicos; o não moderado é mais escalável e permite testar com mais usuários em menos tempo.

2. Como as diretrizes da WCAG 2.1 influenciam as decisões de design?

As diretrizes da WCAG 2.1 definem critérios mínimos de acessibilidade que orientam escolhas como contraste de cores, navegação por teclado, uso de textos alternativos e estrutura semântica do HTML. Elas funcionam como uma referência técnica que integra acessibilidade desde as primeiras etapas do design, evitando retrabalho.

3. Por que incluir usuários com deficiência é mais eficaz do que depender apenas de ferramentas automáticas?

Ferramentas automáticas detectam erros técnicos (como ausência de atributos alt), mas não conseguem avaliar a experiência real de uso. Usuários com deficiência identificam barreiras contextuais, fluxos confusos e problemas de compreensão que só aparecem na prática.

4. De que forma o card sorting e o teste de árvore se complementam?

O card sorting ajuda a descobrir como os usuários agrupam e nomeiam informações, sendo útil na criação da estrutura de navegação. O teste de árvore valida essa estrutura, verificando se os usuários conseguem encontrar o que buscam dentro da hierarquia definida. Juntos, formam um ciclo de criação e validação da arquitetura de informação.

5. Quais são os principais indicadores de fricção em uma interface?

Os principais indicadores são:

- Hesitação ou pausas longas antes de agir
- Erros frequentes ao interagir com elementos
- Comentários de confusão ou frustração durante o teste
- Abandono da tarefa antes de concluí-la
- Necessidade de retornar a etapas anteriores repetidamente

Exemplos de Testes

1. Teste de Usabilidade: Fluxo de Compra

Cenário

A usuária Ana, 34 anos, quer comprar um tênis em um e-commerce pelo celular pela primeira vez, sem nunca ter usado o site.

Como testar

Peça que ela encontre um tênis, adicione ao carrinho e finalize a compra até a tela de confirmação, sem instruções sobre como navegar. Observe sem interferir e peça que ela verbalize o que está pensando em cada etapa.

Objetivo

Identificar se o fluxo de compra é intuitivo, se há pontos de abandono, se os botões de ação estão claros e se o processo de checkout gera dúvidas ou insegurança.

2. Teste de Acessibilidade: Navegação por Teclado

Cenário

Um usuário com limitação motora nos membros superiores utiliza apenas o teclado para navegar e precisa preencher um formulário de cadastro em um portal de serviços públicos.

Como testar

Desconecte o mouse e tente percorrer toda a interface usando somente Tab, Shift+Tab, Enter e as teclas de seta. Verifique se todos os campos recebem foco, se a ordem de navegação é lógica e se o foco está sempre visível.

Objetivo

Garantir que nenhum campo, botão ou link seja inacessível via teclado e que o indicador de foco esteja sempre visível.

3. Teste de Acessibilidade: Leitor de Tela em Formulário

Cenário

Um usuário cego utiliza o leitor de tela NVDA para tentar criar uma conta em um aplicativo de banco digital, preenchendo nome, CPF, e-mail e senha.

Como testar

Ative o NVDA e navegue pelo formulário apenas ouvindo o que o leitor anuncia em cada campo. Verifique se os labels estão associados corretamente aos inputs, se as mensagens de erro são lidas automaticamente e se o botão de envio tem nome descritivo.

Objetivo

Assegurar que o formulário seja completamente compreensível por áudio, sem depender de nenhuma referência visual, garantindo que o usuário consiga se cadastrar de forma autônoma.

Fundamentos de Segurança da Informação nos Testes

O que é?

A aplicação de práticas de segurança no processo de testes de software (QA), com foco na identificação e prevenção de vulnerabilidades. Envolve o mapeamento de falhas na aplicação, verificando se o sistema protege os dados do usuário, possui mecanismos de autenticação seguros e controla corretamente o acesso.

Também inclui a validação das entradas de dados para evitar ataques como injeções de código malicioso. O objetivo é garantir que o sistema seja não apenas funcional, mas também seguro e confiável.

Por que aplicar?

A segurança nos testes é essencial para proteger dados sensíveis, evitar falhas que possam ser exploradas por invasores e garantir a confiança do usuário no sistema. Identificar vulnerabilidades antes da produção reduz riscos, prejuízos financeiros e danos à reputação da aplicação.

Como aplicar nos projetos?

A segurança nos testes é aplicada por meio da análise de vulnerabilidades comuns, como falhas de autenticação, controle de acesso e validação de entradas. Isso inclui testar se os dados estão protegidos, se apenas usuários autorizados têm acesso ao sistema e se as entradas estão seguras contra ataques como injeção de código. Técnicas como testes de vulnerabilidade e simulação de ataques (pentests) são utilizadas para identificar possíveis falhas.

O que é a OWASP?

A OWASP (Open Worldwide Application Security Project) é uma organização internacional sem fins lucrativos que tem como objetivo melhorar a segurança de aplicações de software. Ela fornece materiais, boas práticas e ferramentas gratuitas para ajudar desenvolvedores e testadores a criar sistemas mais seguros.

Um dos seus projetos mais conhecidos é o OWASP Top 10, uma lista das dez vulnerabilidades mais comuns e críticas em aplicações web. Essa lista serve como guia de referência para identificar e prevenir falhas como injeção de código, falhas de autenticação e exposição de dados sensíveis.

Perguntas e Respostas

1. Por que a segurança é importante nos testes de software?

Porque falhas de segurança podem expor dados sensíveis dos usuários, permitir acessos não autorizados e causar danos financeiros e de reputação à aplicação e à empresa.

2. O que pode acontecer se um sistema não for testado em termos de segurança?

O sistema pode ser vulnerável a ataques como injeção de código, acesso indevido a dados e exploração de falhas de autenticação, comprometendo a integridade e a confiança na aplicação.

3. Como sabemos se um sistema protege bem os dados do usuário?

Testando se dados sensíveis estão criptografados, se o acesso é restrito a usuários autorizados e se as informações não são expostas indevidamente em logs ou respostas da API.

4. Por que validar as entradas de dados é importante?

Porque entradas maliciosas podem ser usadas para executar ataques como SQL Injection e XSS, comprometendo o banco de dados e a segurança dos usuários.

5. O que são testes de segurança e como eles ajudam a evitar ataques?

São testes que simulam ataques reais (pentests) e analisam vulnerabilidades do sistema. Eles ajudam a identificar e corrigir falhas antes que invasores possam explorá-las em produção.

Exemplos de Testes

1. Login do Sistema

Cenário

Um usuário tenta fazer login na aplicação.

Como testar

Inserir senha errada várias vezes, testar senhas fracas e verificar se há bloqueio após tentativas excessivas.

Objetivo

Garantir que o sistema tenha autenticação segura e proteção contra acessos indevidos.

2. Formulário de Cadastro

Cenário

Usuário preenche um formulário com nome, e-mail e senha.

Como testar

Inserir códigos maliciosos nos campos (ex: scripts ou comandos SQL).

Objetivo

Verificar se o sistema está protegido contra injeção de código (SQL Injection e XSS).

3. Acesso a Dados do Usuário

Cenário

Um usuário logado tenta acessar dados de outro usuário.

Como testar

Alterar URLs ou requisições para tentar visualizar informações de outra conta.

Objetivo

Garantir que o sistema tenha controle de acesso correto e proteja os dados de cada usuário.

Perguntas escolhidas

Testes Não Funcionais (Performance, Carga, Estresse)

-> 1. Qual é a diferença prática entre avaliar o tempo de resposta em testes de performance, o comportamento no limite esperado em testes de carga e o comportamento até a quebra do sistema em testes de estresse?

R: Os testes de performance, carga e estresse avaliam a robustez do software, mas focam em cenários diferentes: performance verifica a velocidade geral, carga simula o uso diário normal, e estresse leva o sistema ao limite para identificar pontos de ruptura. Entenda a diferença:

Testes de Usabilidade e Acessibilidade Digital (UX)

-> 5. Quais são as principais diferenças, vantagens e desvantagens entre realizar um teste de usabilidade com usuários reais em comparação com um teste heurístico?

R: O **teste com usuários reais** serve para ver onde as pessoas travam de verdade e entender a frustração delas na prática. Ele é profundo e realista, mas demora mais.

Já a avaliação heurística é uma revisão rápida feita por especialistas para checar se o design segue as regras básicas. É barata e ótima para achar erros óbvios, mas não substitui a visão do cliente.

Fundamentos de Segurança da Informação nos Testes

-> 10. Qual é a diferença conceitual e prática entre as abordagens de avaliação de segurança SAST, DAST
e o Pentest (teste de intrusão) manual?

R: Resumidamente: SAST analisa o código (desenvolvimento), DAST testa o aplicativo em execução (homologação/produção) e o Pentest Manual simula ataques reais (validação final/completa).